IOT-BASED INTELLIGENT ACCIDENT DETECTION & EMERGENCY RESPONSE SYSTEM

A PROJECT REPORT

Submitted by

Mirudhula R	**2303717610422100**
Mohammed Waseem M	**2303717610421101**
Nandhakumar D	**2303717610421102**
Navadeep T	**2303717610421103**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

COIMBATORE-641014

ANNA UNIVERSITY: CHENNAI 600 025

April 2026

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	**INTRODUCTION**	1
2	**COMPONENTS REQUIRED**	2
	2.1 SOFTWARE	2
	2.2 HARDWARE	3
3	**SYSTEM DESIGN AND IMPLEMENTATION**	4
	3.1 ARCHITECTURE DIAGRAM	4
	3.2 FLOWCHART	5
	3.3 CIRCUIT DIAGRAM COMPONENTS	6
	3.4 CONNECTION CONFIGURATION	6
	3.5 WORKING PRINCIPLE	7
4	**SOURCE CODE**	8
5	**WORKING AND RESULTS**	12
6	**CONCLUSION**	14

CHAPTER 1
INTRODUCTION

1.1 PROBLEM STATEMENT

Road accidents are a major cause of injuries and fatalities worldwide. One of the primary reasons for increased death rates is the delay in providing medical assistance to accident victims. In many situations, victims are unable to communicate due to unconsciousness or severe injuries. Additionally, lack of proper location information further delays emergency response.

Hence, there is a need for an automated system that can detect accidents instantly, identify the severity using multiple parameters, and communicate the exact location to emergency contacts without human intervention. An IoT-based intelligent system can effectively address this issue by ensuring quick detection and faster response.

1.2 OBJECTIVE

The main objective of this project is to design and develop an IoT-based intelligent accident detection and emergency response system that enhances road safety and reduces emergency response time.

The specific objectives are:

- To automatically detect accidents using multiple sensors
- To accurately determine the location of the accident
- To provide a user confirmation mechanism with a 10–15 second timer to avoid false alerts
- To automatically send emergency alerts with location details if no response is received
- To ensure quick communication with predefined contacts or emergency services
- To improve the chances of timely medical assistance and reduce fatalities

CHAPTER 2

COMPONENTS REQUIRED

2.1 SOFTWARE

The software components used in this project include:

- C++ (Arduino IDE) – Used for programming the ESP8266 microcontroller and implementing the intelligent accident detection logic based on multi-sensor threshold analysis.
- Arduino IDE – Development environment used to write, compile, debug, and upload the code to the ESP8266 NodeMCU.
- ESP8266 WiFi Library (ESP8266WiFi.h) – Enables WiFi connectivity for real-time internet communication and Telegram message transmission.
- WiFiClientSecure Library – Provides secure HTTPS communication with Telegram Bot API servers for encrypted alert messaging.
- Time Library (time.h) – Used to fetch real-time timestamps from NTP servers and include the exact accident time in the emergency alert.
- Wire Library (Wire.h) – Used for I2C communication between the ESP8266 and the MPU6050 sensor module.
- MPU6050 Library – Used to acquire acceleration values and compute tilt angle for accurate accident detection.
- Telegram Bot API – Used to send accident alert messages, including time and location details, to emergency contacts through Telegram.

These software tools support sensor data acquisition, threshold-based decision logic, real time communication, user confirmation timing, and emergency alert transmission.

2.2 HARDWARE

The hardware components used in this system are:

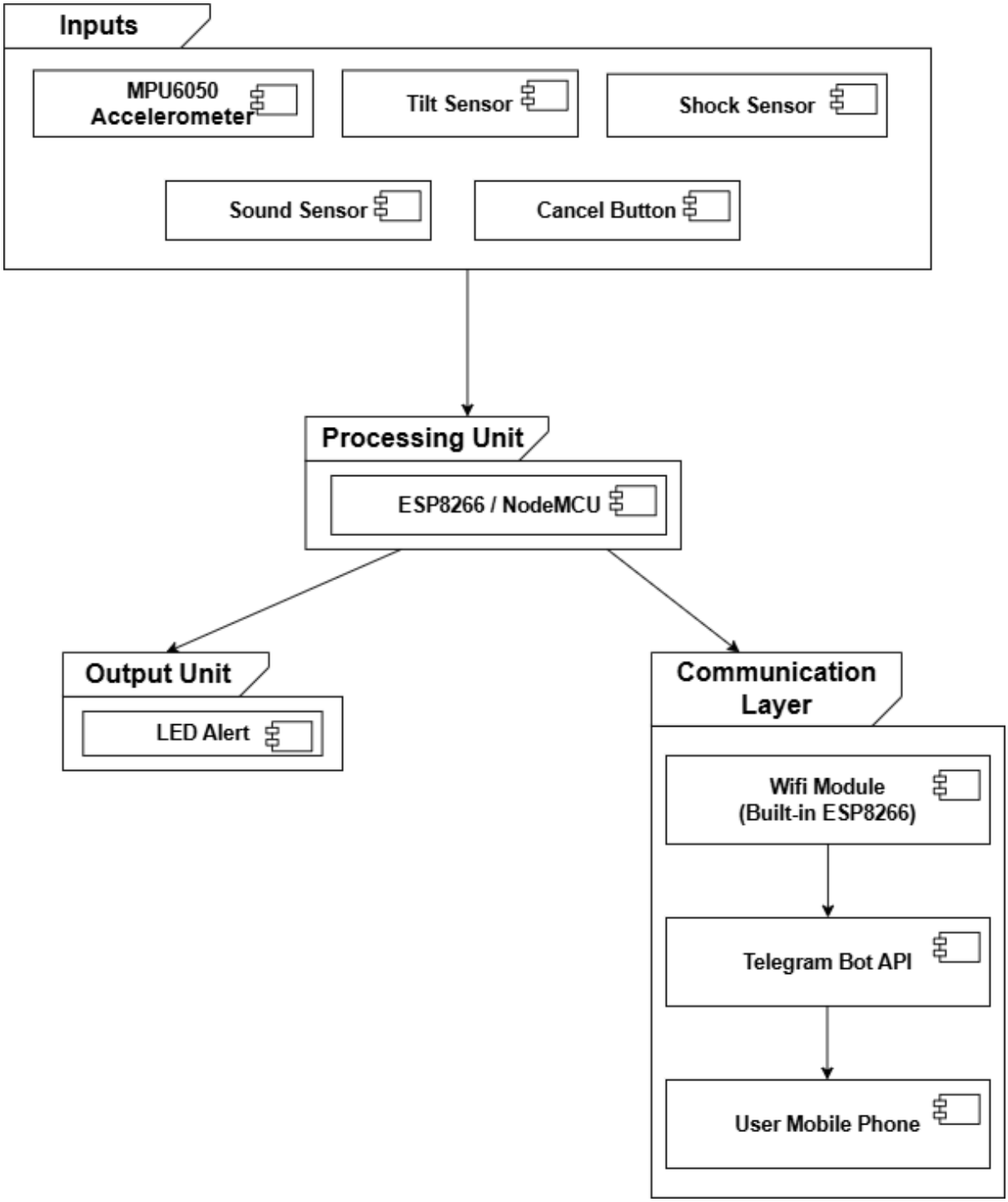
- ESP8266 (NodeMCU) – Acts as the main controller with built-in WiFi capability and performs sensor fusion, accident logic processing, and alert transmission.
- Shock Sensor – Detects sudden physical impact or collision force during an accident.
- Sound Sensor – Detects high-intensity collision sound or crash noise.
- Tilt Sensor – Detects abnormal tilt or vehicle rollover
- MPU6050 – Used to measure acceleration and compute vehicle tilt angle, enabling sudden impact and rollover detection.
- Push Button – Allows the user to cancel false accident alerts within 15 seconds before emergency messages are sent.
- LED Indicator – Provides visual indication when an accident is detected and the confirmation timer is active.
- Power Supply / USB Cable – Provides power to the ESP8266 NodeMCU and connected sensors.

This hardware setup enables accurate multi-sensor accident confirmation, false alert reduction, and reliable emergency notification.

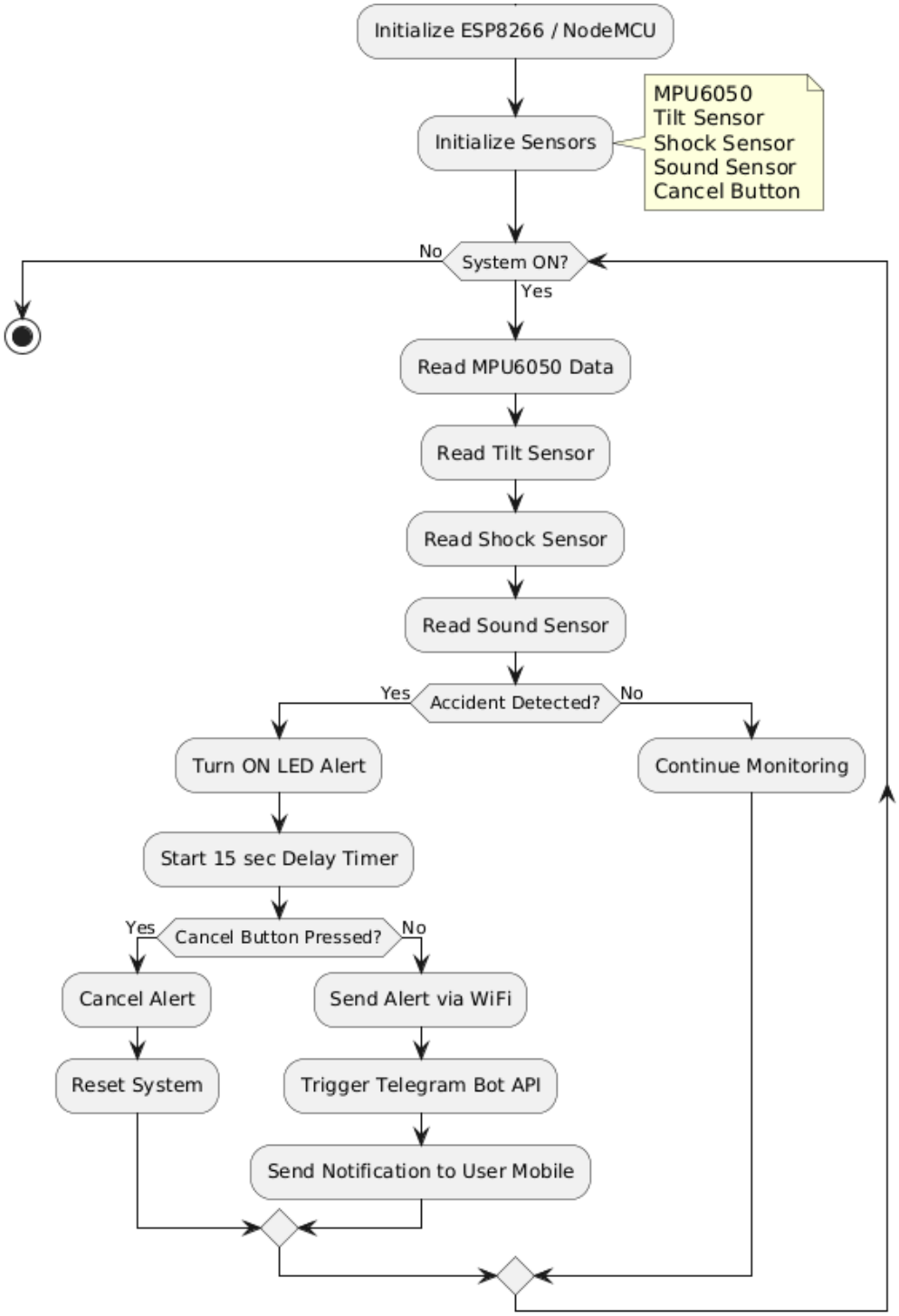
CHAPTER 3

ARCHITECTURE / CIRCUIT DIAGRAM

3.1 ARCHITECTURE DIAGRAM



3.2 FLOWCHART



3.3 CIRCUIT DIAGRAM COMPONENTS

Component Name	Brief Description
MPU6050 Accelerometer	A motion sensor module that contains a 3-axis accelerometer + 3-axis gyroscope
Tilt Sensor	A switch-based sensor that changes state when tilted beyond a certain angle
Shock Sensor	A vibration/impact sensing module
Sound Sensor	Sound detection module with digital output
ESP8266 (NodeMCU)	Wi-Fi enabled microcontroller board
Push Button	Manual input switch
LED Indicator	Light indicator component

3.4 CONNECTION CONFIGURATION

MPU6050 → I2C Interface

- VCC → 3.3V
- GND → GND
- SDA → D2 (GPIO4)
- SCL → D1 (GPIO5)

Shock Sensor (KY-002)

- VCC → 3.3V
- GND → GND
- Signal (S) → D3 (GPIO0)

Tilt Sensor (KY-020)

- VCC → 3.3V
- GND → GND
- Signal (S) → D4 (GPIO2)

Sound Sensor (KY-038)

- VCC → 3.3V
- GND → GND
- D0 → D5 (GPIO14)

Push Button

- One terminal → D6 (GPIO12)
- Other terminal → GND

LED

- Anode (+) → D7 (GPIO13) through 220Ω resistor
- Cathode (–) → GND

3.5 WORKING PRINCIPLE

The system is built using the ESP8266 (NodeMCU) as the central controller, which continuously monitors vehicle conditions using multiple sensors.

A shock sensor detects sudden impacts, while a sound sensor identifies high noise levels during a collision. The MPU6050 measures acceleration and tilt angle to detect sudden deceleration and vehicle rollover.

All sensor data is processed using a multi-sensor threshold approach, where accident detection is based on a combination of conditions rather than a single sensor. Each abnormal reading contributes to an accident score, and when the score exceeds a set threshold, the system confirms an accident.

Once detected, a 15-second confirmation timer is initiated, allowing the user to cancel the alert using a push button. If cancelled, the system stops and requires a manual reset.

If no response is received, the system sends an emergency alert via WiFi using the Telegram Bot API, including the accident location and time obtained from an NTP server. The alert is sent multiple times to ensure delivery, and an LED indicator provides a local visual alert.

After the alert process, the system enters a manual lock state, preventing further automatic operation until restarted. This ensures reliability and reduces false alerts.

CHAPTER 4

SOURCE CODE

```
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>
#include <time.h>
#include <Wire.h>
#include <math.h>
// ---------------- WIFI ----------------
const char* ssid = "MI";
const char* password = "12345678";
// ---------------- TELEGRAM ----------------
String botToken = "8641117658:AAEFOlVjo85IqUr6sCUzZG2bEan52L39S00";
String chatID = "1792879382";
// ---------------- PINS ----------------
const int shockPin = A0;
const int soundPin = D5;
const int buttonPin = D6;   // CHANGED from D0 to D6
const int ledPin = D7;
// ---------------- MPU6050 ----------------
const int MPU_ADDR = 0x68;
// ---------------- VARIABLES ----------------
WiFiClientSecure client;
bool accidentTriggered = false;
bool systemLocked = false;
float prevAccel = 1.0;
int startupCounter = 0;
int lockMessageCount = 0;
// ---------------- TIME ----------------
const char* ntpServer = "pool.ntp.org";
long gmtOffset_sec = 19800;
int daylightOffset_sec = 0;
// ---------------- TIME FUNCTION ----------------
String getTime() {
  time_t now = time(nullptr);
  struct tm* timeinfo = localtime(&now);
  char buffer[20];
  sprintf(buffer, "%02d:%02d:%02d",
          timeinfo->tm_hour,
          timeinfo->tm_min,
          timeinfo->tm_sec);
  return String(buffer);  }
// ---------------- TELEGRAM ----------------
void sendTelegram(String message) {
  client.setInsecure();
  if (!client.connect("api.telegram.org", 443)) {
    Serial.println("Telegram connection failed");
    return;  }
```

```
  String url = "/bot" + botToken +
               "/sendMessage?chat_id=" + chatID +
               "&text=" + message;
  client.print(String("GET ") + url + " HTTP/1.1\r\n" +
               "Host: api.telegram.org\r\n" +
               "Connection: close\r\n\r\n");
  delay(100);
  client.stop();
  Serial.println(">>> TELEGRAM ALERT SENT <<<");  }
// ---------------- BUTTON ----------------
bool isButtonPressed() {
  return digitalRead(buttonPin) == LOW;   }
// ---------------- MPU READ ----------------
void readMPU(int16_t &ax, int16_t &ay, int16_t &az) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, 6, true);
  ax = Wire.read() << 8 | Wire.read();
  ay = Wire.read() << 8 | Wire.read();
  az = Wire.read() << 8 | Wire.read();   }
// ---------------- SETUP ----------------
void setup() {
  Serial.begin(115200);
  pinMode(shockPin, INPUT);
  pinMode(soundPin, INPUT);
  // FIXED: internal pull-up to avoid floating
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
  Wire.begin(D2, D1);
  // Wake MPU6050
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission(true);
  WiFi.begin(ssid, password);
  Serial.print("Connecting WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");   }
  Serial.println("\nWiFi Connected");
  configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
  delay(3000);  }
// ---------------- LOOP ----------------
void loop() {
  // ---------------- LOCK MODE ----------------
  if (systemLocked) {
    if (lockMessageCount < 2) {
      Serial.println(">>> SYSTEM LOCKED <<<");
```

```
    Serial.println("Press button to reset");
    lockMessageCount++;  }
  if (isButtonPressed()) {
    systemLocked = false;
    accidentTriggered = false;
    lockMessageCount = 0;
    digitalWrite(ledPin, LOW);
    Serial.println(">>> SYSTEM RESET <<<");
    delay(2000);   }
  delay(300);
  return;    }
// ---------------- STARTUP IGNORE ----------------
if (startupCounter < 3) {
  startupCounter++;
  delay(1000);
  return;   }
// ---------------- SENSOR READINGS ----------------
int shockValue = analogRead(shockPin);
int soundState = digitalRead(soundPin);
int16_t ax, ay, az;
readMPU(ax, ay, az);
float currentAccel = sqrt(
    (float)ax * ax +
    (float)ay * ay +
    (float)az * az
) / 16384.0;
float accelChange = abs(currentAccel - prevAccel);
prevAccel = currentAccel;
float tiltAngle = atan2((float)ay, (float)az) * 180.0 / PI;
String soundStatus = (soundState == LOW) ? "DETECTED" : "NORMAL";
Serial.println("\n------ SENSOR VALUES ------");
Serial.print("Shock: "); Serial.println(shockValue);
Serial.print("Sound: "); Serial.println(soundStatus);
Serial.print("Accel Change: "); Serial.println(accelChange);
Serial.print("Tilt Angle: "); Serial.println(tiltAngle);
int score = 0;
// Better thresholds
if (shockValue > 50) score++;
if (soundState == LOW) score++;
if (accelChange > 1.0) score++;
if (abs(tiltAngle) > 25) score++;
Serial.print("Score: ");
Serial.println(score);
// ---------------- ACCIDENT DETECTION ----------------
if (score >= 2 && !accidentTriggered) {
  accidentTriggered = true;
  digitalWrite(ledPin, HIGH);
  Serial.println(">>> ACCIDENT DETECTED <<<");
  bool cancelled = false;
  unsigned long startTime = millis();
```

```
    unsigned long lastPrint = 0;
    while (millis() - startTime < 15000) {
      int remaining = 15 - ((millis() - startTime) / 1000);
      if (millis() - lastPrint >= 1000) {
        Serial.print("Cancel within ");
        Serial.print(remaining);
        Serial.println(" sec");
        lastPrint = millis();   }
      if (isButtonPressed()) {
        cancelled = true;
        Serial.println(">>> ALERT CANCELLED <<<");
        break;   }
      delay(50);
      yield();     }
    if (!cancelled) {
      String msg = "Accident Detected!\n";
      msg += "Location: https://maps.google.com/?q=13.0827,80.2707\n";
      msg += "Time: " + getTime();
      msg.replace(" ", "%20");
      msg.replace("\n", "%0A");
      sendTelegram(msg);   }
    digitalWrite(ledPin, LOW);
    systemLocked = true;
    Serial.println(">>> SYSTEM LOCKED <<<");  }
  // 5 sec pause for easy monitoring
  delay(5000);
}
```

CHAPTER 5

WORKING & RESULTS

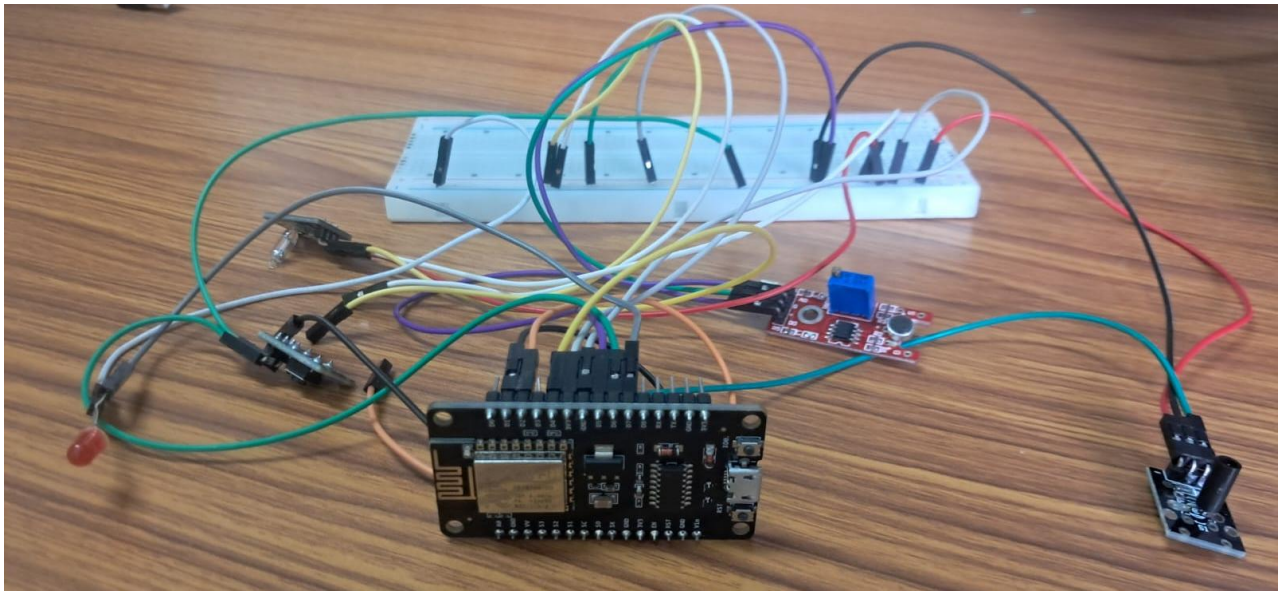
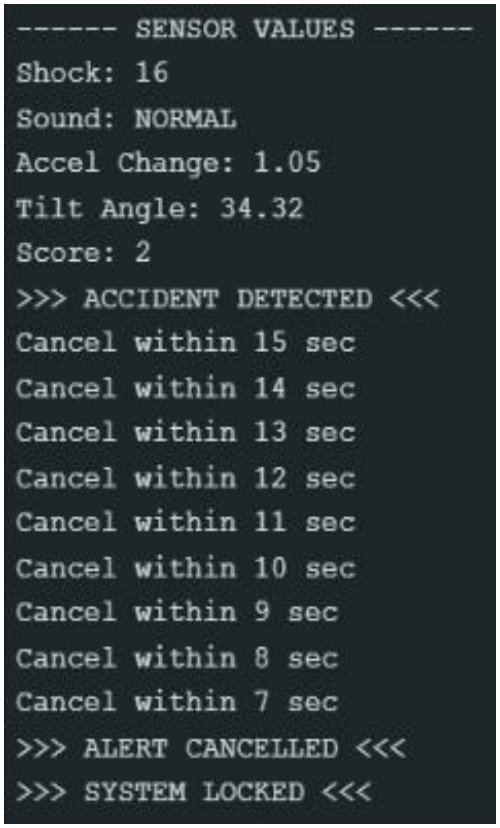


Figure 1: Working model of Accident Detection



```
------ SENSOR VALUES ------
Shock: 16
Sound: NORMAL
Accel Change: 1.05
Tilt Angle: 34.32
Score: 2
>>> ACCIDENT DETECTED <<<
Cancel within 15 sec
Cancel within 14 sec
Cancel within 13 sec
Cancel within 12 sec
Cancel within 11 sec
Cancel within 10 sec
Cancel within 9 sec
Cancel within 8 sec
Cancel within 7 sec
>>> ALERT CANCELLED <<<
>>> SYSTEM LOCKED <<<
```

Figure 2: Accident Detected and Cancelled within 15 seconds

```
------ SENSOR VALUES ------
Shock: 17
Sound: NORMAL
Accel Change: 1.25
Tilt Angle: -135.00
Score: 2
>>> ACCIDENT DETECTED <<<
Cancel within 15 sec
Cancel within 14 sec
Cancel within 13 sec
Cancel within 12 sec
Cancel within 11 sec
Cancel within 10 sec
Cancel within 9 sec
Cancel within 8 sec
Cancel within 7 sec
Cancel within 6 sec
Cancel within 5 sec
Cancel within 4 sec
Cancel within 3 sec
Cancel within 2 sec
Cancel within 1 sec
>>> TELEGRAM ALERT SENT <<<
>>> SYSTEM LOCKED <<<
```

Figure 3: Accident Detected and alert sent to telegram bot

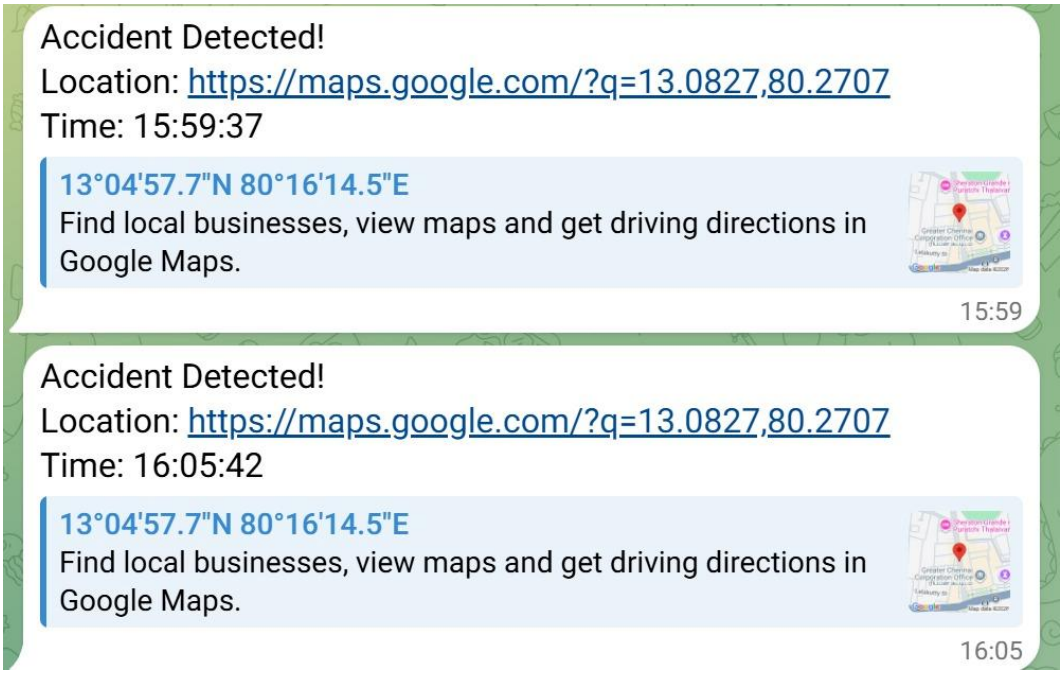


Figure 4: Telegram Bot Alert

CHAPTER 6

CONCLUSION

The IoT-based Accident Detection and Alert System was successfully designed and implemented using the ESP8266 NodeMCU and multiple sensors such as the MPU6050 accelerometer, tilt sensor, shock sensor, and sound sensor. The system continuously monitors sudden changes in motion, impact, tilt angle, and sound levels to accurately detect accident conditions.

By integrating Wi-Fi communication with the Telegram Bot API, the project enables real-time alert transmission to the user's mobile phone, ensuring immediate notification during emergencies. The inclusion of a manual cancel button and a short delay window helps reduce false alerts by allowing the user to cancel accidental triggers.

This project demonstrates how IoT technology can be effectively used for smart safety and emergency response applications. The system is cost-effective, reliable, and suitable for real-world vehicle safety monitoring, helping to improve response time during accidents and potentially save lives.